

Задача 2. Умный роутинг выплат

Полное описание

Вам необходимо разработать механизм умного распределения выплат между несколькими платёжными партнерами. Для каждой новой операции система должна выбирать наиболее подходящего партнера с учетом стратегии и правил. Если выбранный партнёр не может провести выплату или отказывает в обработке, необходимо автоматически перейти к следующему подходящему варианту.

Для каждой выплаты необходимо сохранять понятное объяснение принятого решения: какие варианты рассматривались, почему был выбран конкретный партнер и по каким причинам остальные были исключены. Также на основе истории операций нужно оценивать качество текущего роутинга и давать рекомендации.



Вводные данные

Все файлы в папке data/:

Файл	Описание
providers.json	Состояние провайдеров на момент симуляции (лимиты, метрики, банки)
operations_history.csv	100 исторических решений роутинга заявок. Их можно использовать для анализа конверсии провайдера, долей count/volume и калибровки
operations_queue.json	пример 10 заявок для роута
sample_routing_decisions.json	Пример формата в котором нужно отдать ответ для автопроверки

`validate.rb` Скрипт автопроверки, именно им будет проверяться ваш итоговый `routing_decisions.json` (для `routing_decisions_test.json` будет схожий скрипт)

Формат результата роутинга

```
[
  {
    "operation_id": "op_103",
    "selected_provider": "quickpay",
    "attempts": [
      {
        "provider": "vipay",
        "decision": "skipped",
        "reason": "amount_exceeds_limit",
        "details": "150000 > limit_amount_max 100000"
      }
    ]
  }
]
```

```
},
{
  "provider": "payflow",
  "decision": "skipped",
  "reason": "amount_exceeds_limit",
  "details": "150000 > limit_amount_max 50000"
},
{
  "provider": "quickpay",
  "decision": "selected",
  "reason": "only_eligible_provider"
}
],
"simulated_result": "approved",
"latency_sec": 28
}
```

Формат провайдера

Ключевые поля из providers.json:

Поле	Описание
traffic_percentage	Целевая доля по количеству заявок
limit_amount_min/max	Диапазон суммы чека
daily_amount_limit / daily_approved_amount	Дневной max / текущий оборот
in_progress_count_limit / in_progress_count	Лимит одновременных заявок
in_progress_amount_limit / in_progress_amount	Лимит суммы in-progress
available_requisites	Свободные реквизиты/терминалы

Поле	Описание
banks / exclude_banks	Фильтр по банку
conversion_24h	Конверсия за 24 часа
provider_margin_pct / merchant_margin_pct	Маржа

Поля, которые командам полезно добавить самим (вывести из истории/поставить субъективные значения, которые будут приближены к реальности):

Поле	Критерий
volume_share_pct	% от объёма
priority	очередь в каскаде
requests_per_minute_limit	интенсивность
daily_turnover_min	фин. обязательство "не менее X ₽/сутки"
daily_turnover_max	фин. обязательство "не более X ₽/сутки" (или использовать daily_amount_limit)

Стратегии распределения

Решение участников должно уметь работать по нескольким стратегиям роутинга, а также по их комбинации

Стратегия	Критерий	Что означает	Пример настройки
-----------	----------	--------------	------------------

1	Процент от количества заявок	Доля по числу операций	vipay 40%, payflow 35%, quickpay 25%
2	Процент от объёма	Доля по сумме (рубли), а не по count	vipay 50% оборота, остальные 50%
3	Очередь в каскаде	Последовательный обход по priority (как депозиты в production)	сначала vipay, при отказе - payflow, затем quickpay
4	По сумме чека	Маршрутизация по диапазону суммы заявки	500–50 000 → payflow; 50 001–100 000 → vipay; >100 000 → quickpay
5	Приоритизация по конверсии	Предпочитать провайдеров с более высокой conversion_24h	soft-boost веса или порядок кандидатов
6	По интенсивности	Rate limit на терминал/провайдера	не более 7 заявок/мин на терминал А, 15/мин на терминал В
7	По фин. обязательствам	Min/max оборот в сутки, если гейт выдан на условиях по обороту	не менее 2 000 000 ₽/сутки на payflow; не более 5 000 000 ₽ на vipay

Правила роутинга

Hard-constraints

Hard-constraints - это условия допуска провайдера к роутингу. Они отвечают на вопрос: “Можно ли вообще отправить эту заявку этому провайдеру?”. Если хотя бы одно жёсткое условие не выполнено, провайдер исключается из рассмотрения независимо от стратегии.

Эти проверки нельзя нарушать независимо от выбранной стратегии:

Проверка	Условие пропуска
Статус провайдера	status != "active"
Диапазон суммы чека	amount вне limit_amount_min/max
Дневной max	daily_approved_amount + amount > daily_amount_limit
In-progress count/amount	превышение лимитов одновременных заявок
Банковский фильтр	банк не подходит под banks / exclude_banks
Маржа	provider_margin_pct > merchant_margin_pct (если нет allow_negative_agreement)
Реквизиты	available_requisites == 0
Интенсивность	превышен requests_per_minute_limit

Fallback: при отказе/таймауте - исключить провайдера из пула и выбрать следующего. Если пул пуст - fallback на **spacepayments** (self-provider).

Обновление метрик: после каждой заявки обновлять загрузку, оборот, счётчики интенсивности выбранного провайдера.

Soft-goals

Soft-goals - это цели распределения заявок между провайдерами. Они отвечают на вопрос: "Кого предпочтительнее выбрать из провайдеров, которые уже прошли Hard-constraints?"

Нарушение soft-goal не запрещает использовать провайдера. Вместо этого система повышает или понижает его приоритет в зависимости от того, насколько текущее состояние отклоняется от целевого.

Soft-goal	Поле	Как влияет на выбор
Целевая доля по количеству заявок	traffic_percentage	Если провайдер получает меньше своей целевой доли - его приоритет повышается; если больше - понижается
Целевая доля по объёму	volume_share_pct	Аналогично, но считается доля от денежного объёма
Конверсия	conversion_24h	Более высокая конверсия может повышать приоритет, если стратегия учитывает конверсию

Позиция в каскаде **priority** Определяет очередность/предпочтительность провайдера при каскадной или priority-стратегии

Минимальный дневной оборот **daily_turnover_min** Если минимум ещё не набран - провайдеру можно повышать приоритет
Согласование правил

Критерии могут конфликтовать. Решение должно:

1. разделять hard-constraints и soft-goals;
2. применять hard-фильтры до выбора стратегии;
3. внутри допустимого пула выбирать кандидата по активной стратегии (или комбинации);
4. при конфликте soft-целей -использовать явный приоритет политик и фиксировать это в **attempts** / отчёте;
5. при отказе выбранного провайдера -fallback на следующего подходящего без нарушения hard-ограничений.

Пример конфликта:

- Цель: 40% заявок на viraу.
- Факт: viraу почти исчерпал дневной max, а на rayflow висит обязательство "не менее 2 млн ₽/сутки".
- Ожидание: роутер снижает долю viraу, дотягивает оборот rayflow, объясняет это в аналитике.

Процесс разработки и оценки

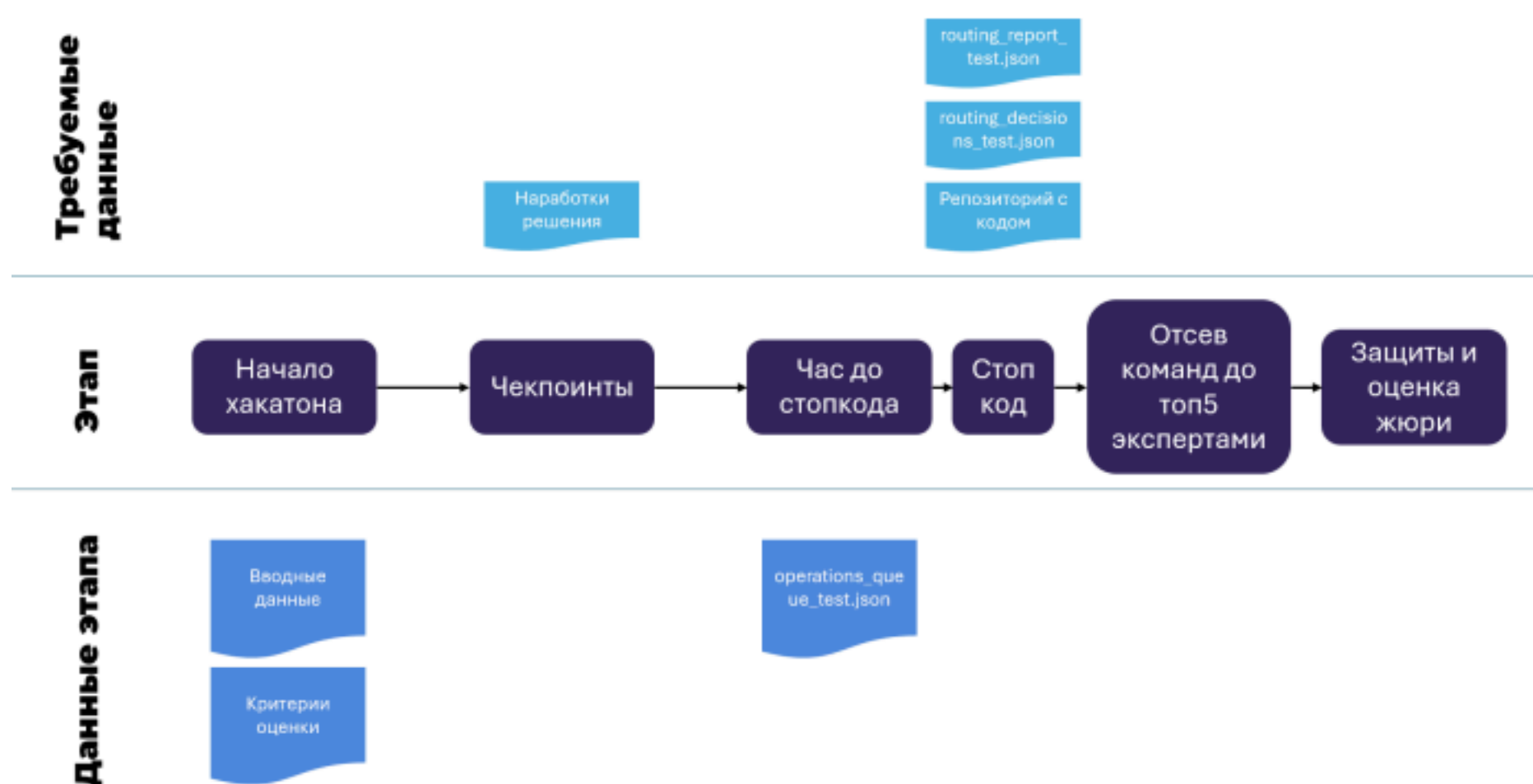
После начала хакатона команды получают вводные данные кейса (см раздел “Вводные данные”) и критерии оценки (см раздел “Критерии”)

За час до стопкода командам будет выдан operations_queue_test.json, команды должны его обработать и предоставить ожидаемый минимальный результат (см раздел “Ожидаемый минимальный результат”)

В течении хакатона у команды будет 3 чекпоинта (встречи с экспертами), по итогу всех трех чекпоинтов эксперты выставляют оценки (максимум 100 баллов) и так формируется топ 5 команд, которые будут допущены до защит

На защитах команд их будут оценивать жюри по своим критериям (см раздел “Критерии”), суммарная оценка команды за хакатон складывается как баллы технического жюри + баллы отраслевого жюри (максимум 160 баллов)

По итогам выставленных баллов формируется лидерборд и отбираются топ3 команды призеры

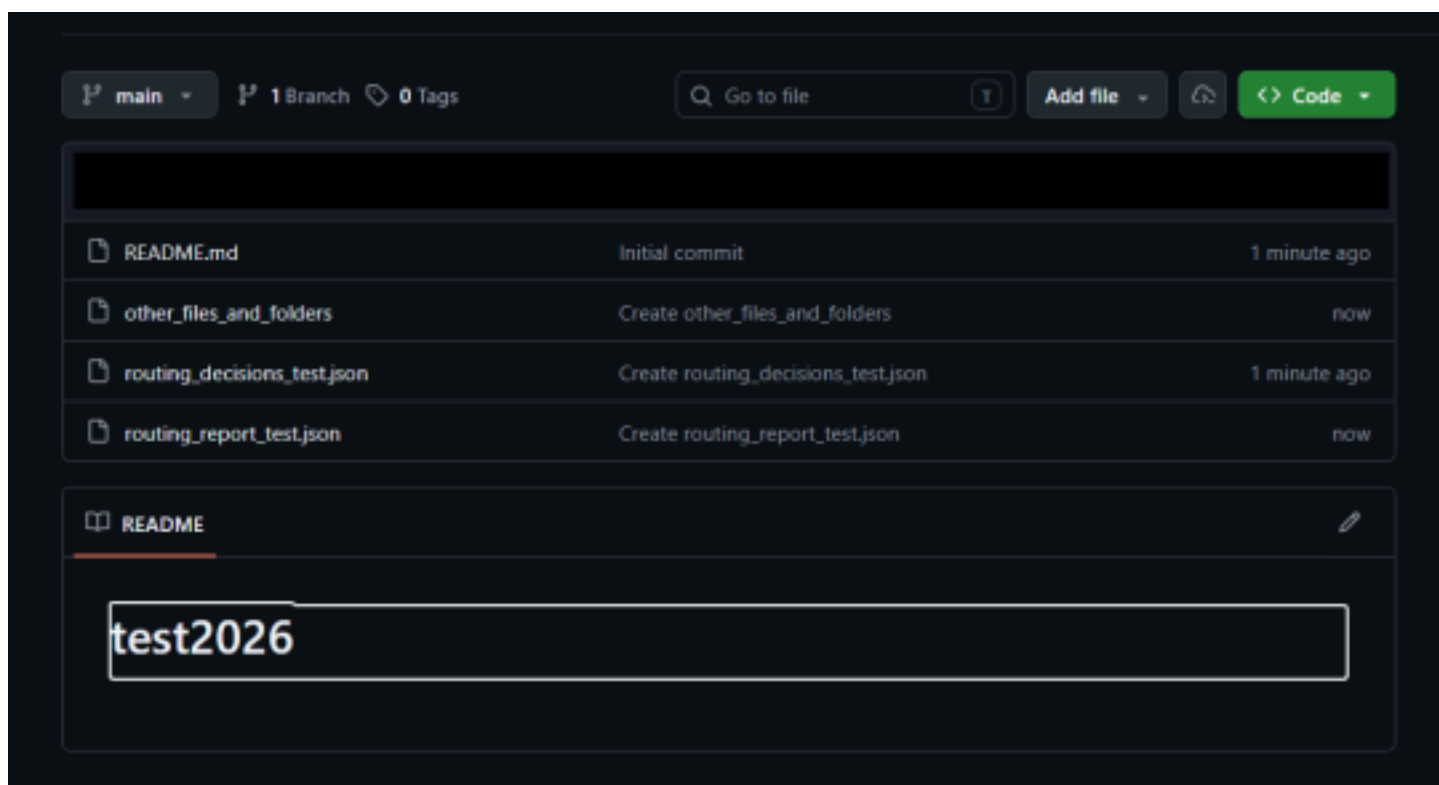


Ограничения

- 1) На кейсе нельзя использовать проприетарные технологии и решения с закрытым исходным кодом.
- 2) Большинство кода внутри репозитория должно быть написано на ruby

За нарушения правил предусматривается дисквалификация

Ожидаемый минимальный результат



Со стороны команд для проверки их решений необходимо предоставить минимум два файла (а также сам код решения):

1. **routing_decisions_test.json** (роутинг платежей для проверки)

За час до стопкода вам будет выдан файл **operations_queue_test.json** (с той же структурой, что и **operations_queue.json**) с новыми заявками на роутинг, его необходимо обработать и распределить заявки по провайдерам, по итогу вы должны сформировать **routing_decisions_test.json** (с той же структурой, что и **routing_decisions.json**) для этих заявок. **routing_decisions_test.json** вы должны положить в корень репозитория с кодом и назвать его "**routing_decisions_test**", должен получиться файл с названием **routing_decisions_test.json**, который будет лежать в корне главного репозитория в ветке main. Этот файл необходим для проверки вашего решения. Если файла нет/он называется неправильно/лежит не там/имеет неправильную структуру, то считается, что команда не прикрепила его.

Структура файла (в примере один роут):

```
[
  {
    "operation_id": "op_103",
    "selected_provider": "quickpay",
    "attempts": [
      {
        "provider": "vipay",
        "decision": "skipped",
        "reason": "amount_exceeds_limit",
        "details": "150000 > limit_amount_max 100000"
      },
      {
        "provider": "payflow",
        "decision": "skipped",
        "reason": "amount_exceeds_limit",
        "details": "150000 > limit_amount_max 50000"
      },
      {
        "provider": "quickpay",
        "decision": "selected",
        "reason": "only_eligible_provider"
      }
    ],
    "simulated_result": "approved",
    "latency_sec": 28
  }
]
```

Обязательные поля:

- operation_id - из входной заявки
- selected_provider - итоговый провайдер
- attempts - массив попыток с provider, decision (selected / skipped), reason
- simulated_result - approved / rejected / expired (можно симулировать по conversion_24h) 2.

routing_report_test.json (итоговая аналитика роутов платежей)

Кроме routing_decisions_test.json вы также должны сформировать routing_report_test.json (его структура представлена ниже), в этом файле вы должны отобразить итог роутинга заявок из operations_queue_test.json, а также рекомендации по дальнейшему роутингу. Файл routing_report_test.json с таким же названием необходимо положить в корне главного репозитория в ветке main. Если файла нет/он называется неправильно/лежит не там/имеет неправильную структуру, то считается, что команда не прикрепила его.

Структура файла (можно добавлять доп поля):

```
{
  "period": "2026-07-30",
  "total_operations": 10,
  "distribution": {
    "vipay": { "count": 2, "share_pct": 20, "target_pct": 40 },
    "payflow": { "count": 2, "share_pct": 20, "target_pct": 35 },
    "quickpay": { "count": 6, "share_pct": 60, "target_pct": 25 }
  },
  "skip_reasons": {
    "amount_exceeds_limit": 3,
    "bank_not_in_list": 5
  },
  "projected_daily_utilization": {
    "vipay": { "used": 3215000, "limit": 5000000, "utilization_pct": 64.3 }
  },
  "recommendations": [
```



```
"payflow близок к дневному лимиту - снизить traffic_percentage"
]
}
```

Критерии

Эксперты

Критерий	Максимум, баллов	Разбалловка
1. Корректность решения задачи	22	12 - при выборе провайдера учитываются ограничения Hard-constraints и Soft-goals; 10 - при отказе предусмотрен переход к следующему доступному провайдеру и fallback.
2. Гибкость маршрутизации	33	18 - используется несколько способов и факторов распределения платежей между провайдерами; 8 - разные факторы маршрутизации могут учитываться совместно при выборе провайдера; 7 - решение не привязано к конкретному набору входных данных и допускает изменение параметров провайдеров и правил.
3. Объяснимость маршрутизации	15	6 - понятно, почему для операции выбран конкретный провайдер; 5 - понятно, почему другие рассматриваемые провайдеры были исключены; 4 - можно проследить последовательность действий при отказе и переходе к следующему провайдеру.
4. Качество технической реализации	20	8 - код структурирован, основные части решения разделены по назначению; 7 - код читаем, названия сущностей и основная логика понятны при просмотре; 5 - предусмотрена обработка основных ошибок и граничных ситуаций.
5. Аналитика и выводы	10	4 - команда показывает основные показатели распределения платежей между провайдерами; 3 - команда анализирует успешность, отказы, загрузку или использование лимитов; 3 - по результатам сформулированы конкретные выводы или предложения по улучшению маршрутизации.
ИТОГО	100	

Жюри

Технические критерии

Критерий	Максимум, баллов	Разбалловка
----------	------------------	-------------

1. Корректность базовой реализации	22	7 - при определении доступных провайдеров соблюдаются Hard-constraints; 7 - при выборе провайдеров соблюдаются Soft-goals; 5 - после обработки операций корректно обновляется состояние провайдеров; 3 - при отказе выполняется следующая попытка, а при отсутствии доступных внешних провайдеров используется fallback.
2. Гибкость правил маршрутизации	32	3 - распределение по количеству операций; 3 - распределение по объему платежей; 3 - распределение по приоритету провайдера; 3 - распределение по диапазонам суммы: диапазон влияет на выбор между доступными провайдерами, а не только используется как обязательное ограничение; 3 - маршрутизация с учетом успешности / конверсии провайдера; 3 - распределение с учетом текущей загрузки: загрузка влияет на выбор между доступными провайдерами, а не только проверяется достижение лимита; 3 - маршрутизация с учетом заданной доли трафика и других бизнес-параметров; 6 - параметры правил можно изменять через настройки или конфигурацию без изменения основной логики; 5 - можно добавлять новые правила и провайдеров без переработки базовой архитектуры.
3. Согласование целевых правил маршрутизации	15	7 - задан способ совместного учета нескольких целевых факторов при выборе среди доступных провайдеров: приоритеты, веса, скоринг или другой формализованный механизм; 4 - определено правило выбора провайдера для случая, когда разные целевые факторы указывают на разных провайдеров; 4 - определено поведение системы, если текущую цель выполнить невозможно: например, требуемая доля трафика относится к недоступному в данный момент провайдеру.

4. Объяснимость маршрутизации 5.	10	4 - для выбранного провайдера указана конкретная причина выбора; 4 - для рассмотренных, но исключенных провайдеров указаны конкретные причины исключения; 2 - сохранена последовательность рассмотрения провайдеров и повторных попыток.
Аналитика качества маршрутизации	11	4 - в отчёте показана фактическая доля операций по каждому провайдеру и её отклонение от заданной доли трафика;

Критерий	Максимум, баллов	Разбалловка
		3 - проанализированы успешность, отказы, загрузка и использование лимитов по провайдерам; 2 - определены причины существенных отклонений от целевого распределения или снижения эффективности; 2 - рекомендации содержат конкретное правило или параметр, который предлагается изменить на основании проведенного анализа.
6. Качество технической реализации	10	4 - архитектура разделяет основные компоненты решения и позволяет вносить изменения без переработки несвязанных частей; 3 - код и структура проекта позволяют понять логику решения и порядок его запуска; 3 - предусмотрена обработка технических ошибок, некорректных входных данных и граничных ситуаций.
7. Наличие требуемых файлов	40	20 - в корне главного репозитория с кодом в ветке main лежит файл routing_decisions_test.json со всеми заявками из operations_queue_test.json в правильной структуре; 20 - в корне главного репозитория с кодом в ветке main лежит файл routing_report_test.json, включающий аналитику роутингов из routing_decisions_test.json и рекомендации в правильной структуре.
ИТОГО	140	

Отраслевые критерии

Критерий	Максимум, баллов	Разбалловка

1. Реализация дополнительных идей		6 - Свободный выбор
2. Выступление команды (умение презентовать результаты своей работы, строить логичный, понятный и интересный рассказ для презентации результатов своей работы)		6 - Хорошее выступление и презентация
3. Полнота проработки решения		8 - Решение выполнено полностью покрывает поставленную задачу
ИТОГО	20	